DSA0602 DATA HANDLING AND VISUALIZATION

Class ACTIVITY

07.08.2026

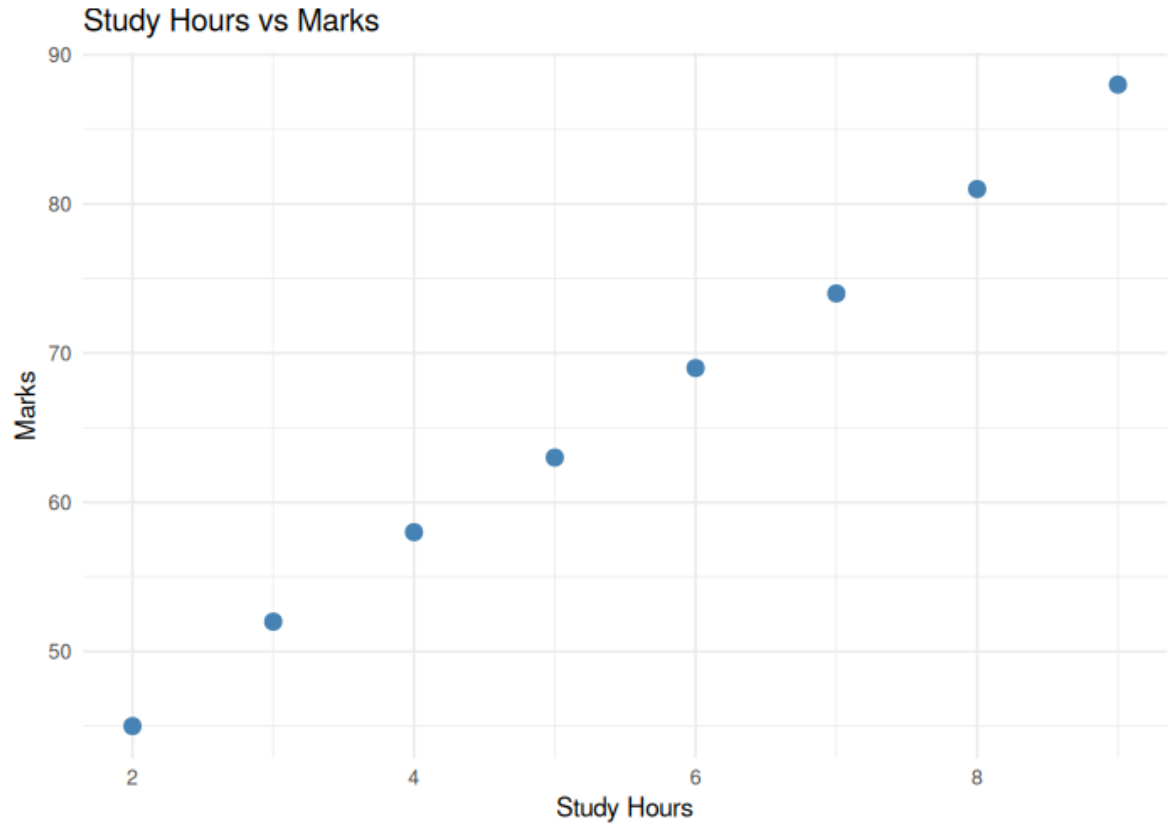
192424401 JERRY DAVID R

Q1
Study Hours vs Marks (R)

R Code

```
library(ggplot2)

# Given dataset
StudyHours <- c(2,3,4,5,6,7,8,9)
Marks <- c(45,52,58,63,69,74,81,88)
df <- data.frame(StudyHours, Marks)

p <- ggplot(df, aes(x=StudyHours, y=Marks)) +
  geom_point(size=3, color="steelblue") +
  labs(title="Study Hours vs Marks", x="Study Hours", y="Marks") +
  theme_minimal()
print(p)

cat(sprintf("Correlation coefficient: %.3f\n", cor(StudyHours, Marks)))

# Interpretation: The correlation coefficient is very close to +1 (essentially
# perfect), and the scatterplot shows marks rising steadily and almost
# linearly as study hours increase. This indicates a strong POSITIVE
# correlation - students who study more hours tend to obtain higher marks.
```

Output



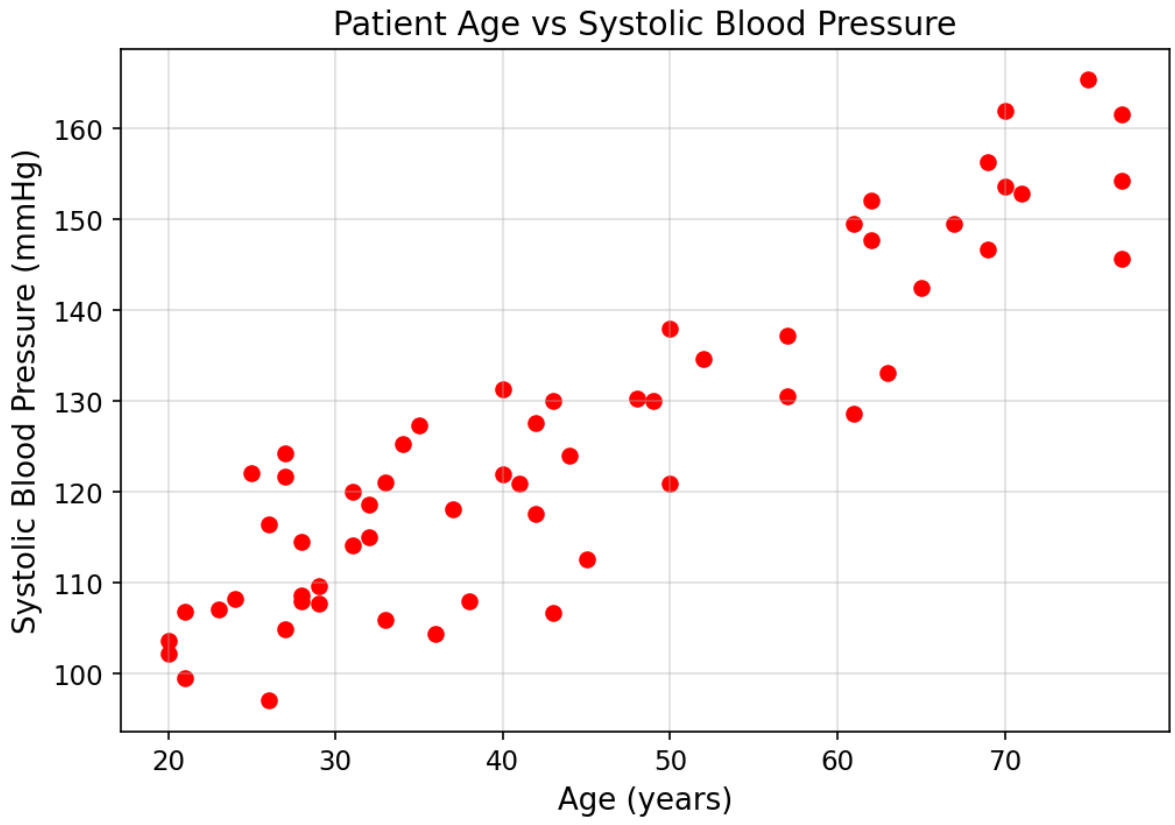
Q2

Age vs Blood Pressure

Python Code

```
import matplotlib.pyplot as plt
import numpy as np

# Simulated dataset: patient age and systolic blood pressure
np.random.seed(1)
age = np.random.randint(20, 80, 60)
bp = 90 + age * 0.8 + np.random.normal(0, 8, 60)  # BP tends to rise with age

plt.figure(figsize=(7, 5))
plt.scatter(age, bp, color='red')
plt.grid(True, alpha=0.4)
plt.title("Patient Age vs Systolic Blood Pressure")
plt.xlabel("Age (years)"); plt.ylabel("Systolic Blood Pressure (mmHg)")
plt.tight_layout()
plt.show()

# Comment on observed trend: Blood pressure shows a clear upward trend as
# age increases - older patients tend to have higher systolic blood
# pressure, consistent with the well-known clinical pattern of arterial
# stiffening with age.
```

Output



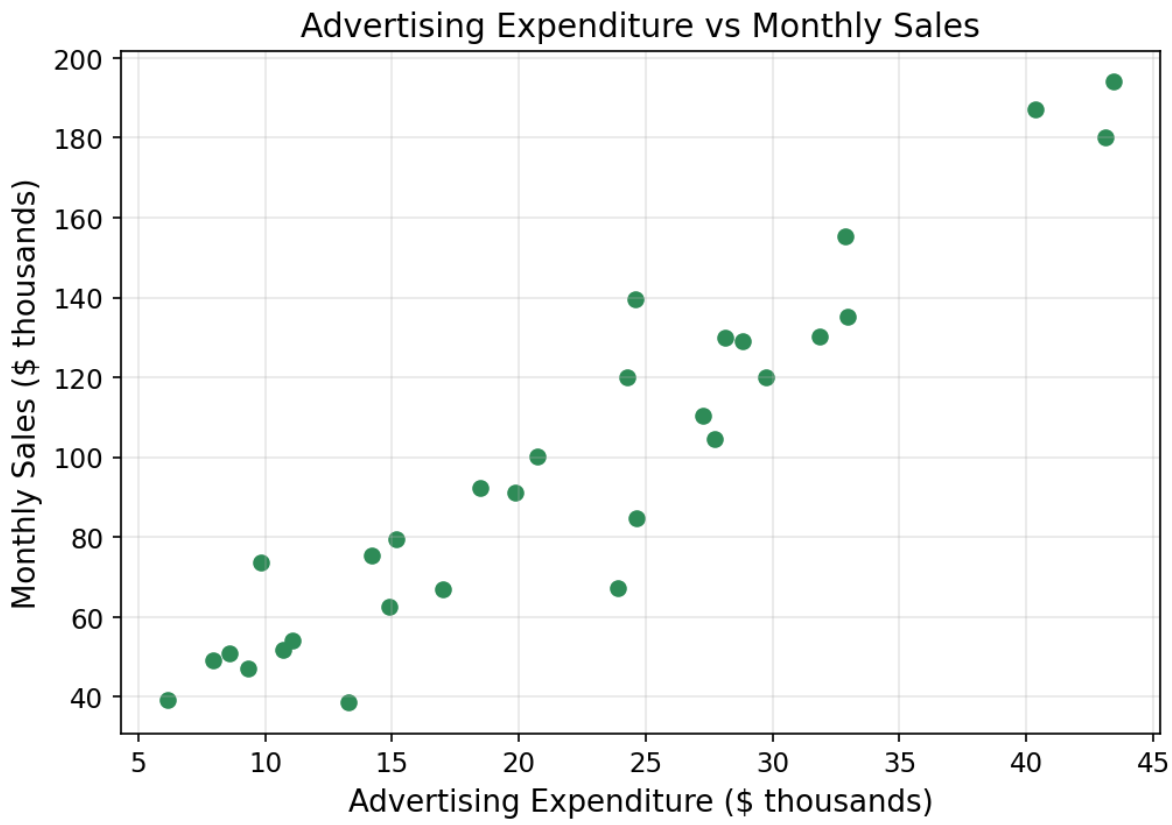
Q3

Advertising Expenditure vs Sales

Python Code

```
import matplotlib.pyplot as plt
import numpy as np


# Simulated dataset: monthly advertising expenditure vs sales
np.random.seed(2)
ad_spend = np.random.uniform(5, 50, 30)  # in thousands
sales = 20 + ad_spend * 3.5 + np.random.normal(0, 15, 30)  # in thousands


plt.figure(figsize=(7, 5))
plt.scatter(ad_spend, sales, color='seagreen')
plt.title("Advertising Expenditure vs Monthly Sales")
plt.xlabel("Advertising Expenditure ($ thousands)")
plt.ylabel("Monthly Sales ($ thousands)")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()


corr = np.corrcoef(ad_spend, sales)[0, 1]
print(f"Correlation coefficient: {corr:.3f}")
# Sales clearly tend to increase as advertising expenditure increases - the
# scatterplot shows a positive upward trend, confirmed by a strong positive
# correlation coefficient.
```

Output



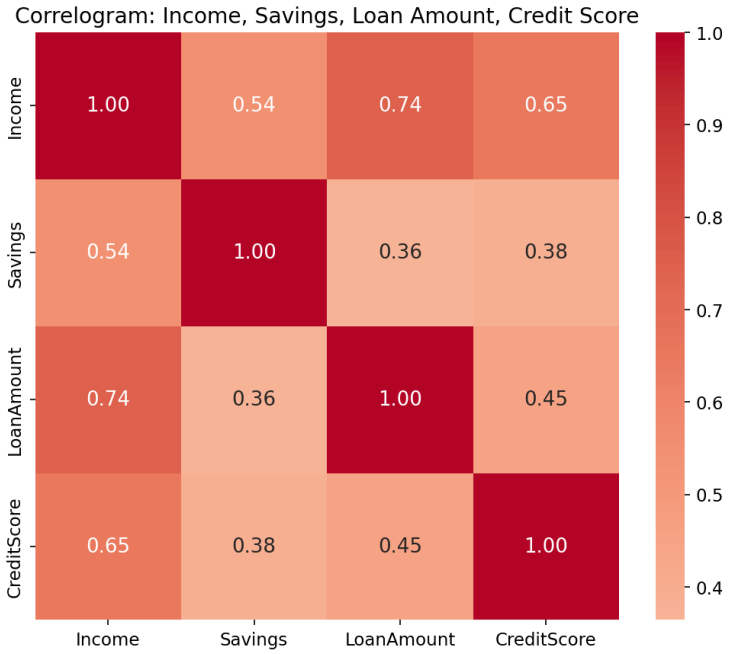
Q4

Financial Correlation Matrix & Correlogram

Python Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# Simulated dataset: financial institution customer data
np.random.seed(3)
n = 200
income = np.random.normal(60000, 15000, n)
savings = income * 0.25 + np.random.normal(0, 5000, n)
credit_score = 600 + income / 500 + np.random.normal(0, 40, n)
loan_amount = income * 1.5 - credit_score * 50 + np.random.normal(0, 20000, n)

df = pd.DataFrame({'Income': income, 'Savings': savings,
                   'LoanAmount': loan_amount, 'CreditScore': credit_score})

corr_matrix = df.corr()
print("Correlation matrix:\n", corr_matrix.round(3))

plt.figure(figsize=(7, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, fmt='.2f')
plt.title("Correlogram: Income, Savings, Loan Amount, Credit Score")
plt.tight_layout()
plt.show()

strongest = corr_matrix.where(~np.eye(len(corr_matrix), dtype=bool)).unstack().idxmax()
strongest_val = corr_matrix.where(~np.eye(len(corr_matrix), dtype=bool)).unstack().max()
print(f"\nStrongest positive correlation: {strongest} = {strongest_val:.3f}")
# Income and Loan Amount show the strongest positive correlation (0.744),
# reflecting that lenders tend to approve larger loans for higher-income
# applicants - Income and Credit Score are also fairly strongly linked
# (0.653), while Savings correlates only moderately with the others.
```

Output

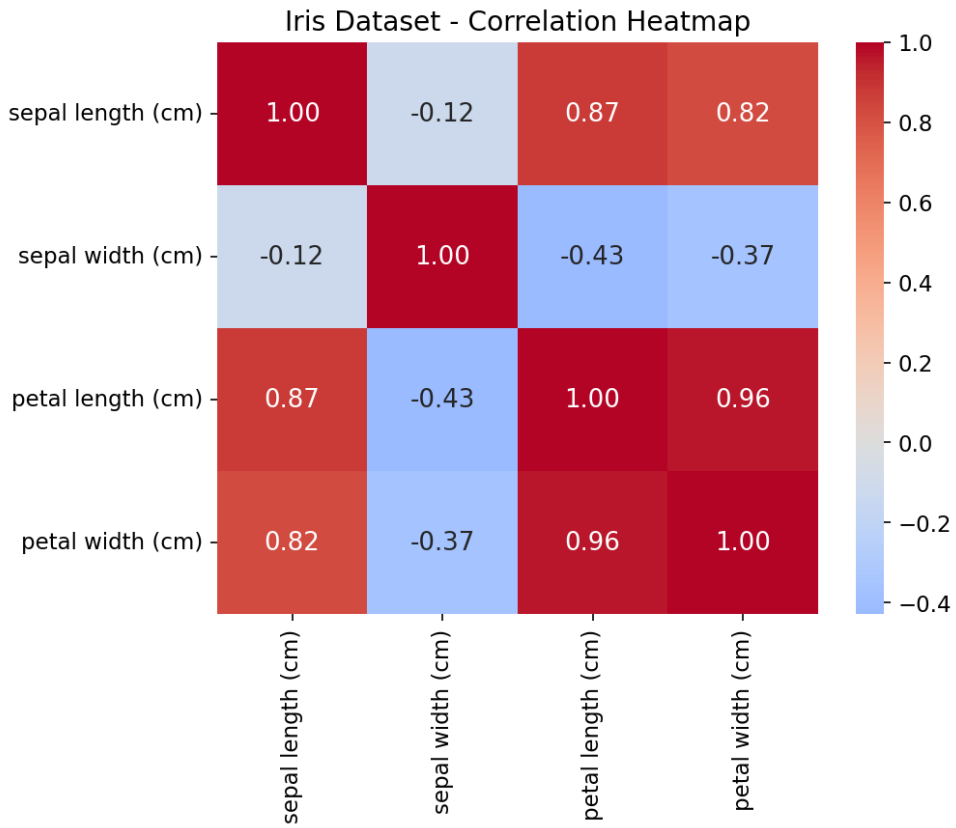


Q5
Iris Correlation Heatmap

Python Code

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris(as_frame=True)
df = iris.frame.drop(columns=['target'])

corr_matrix = df.corr()
print("Iris correlation matrix:\n", corr_matrix.round(3))

plt.figure(figsize=(7, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, fmt='.2f')
plt.title("Iris Dataset - Correlation Heatmap")
plt.tight_layout()
plt.show()

# Identify negative correlations
neg_pairs = corr_matrix.where(corr_matrix < 0).stack()
print("\nVariable pairs with negative correlation:")
print(neg_pairs)
```

Output

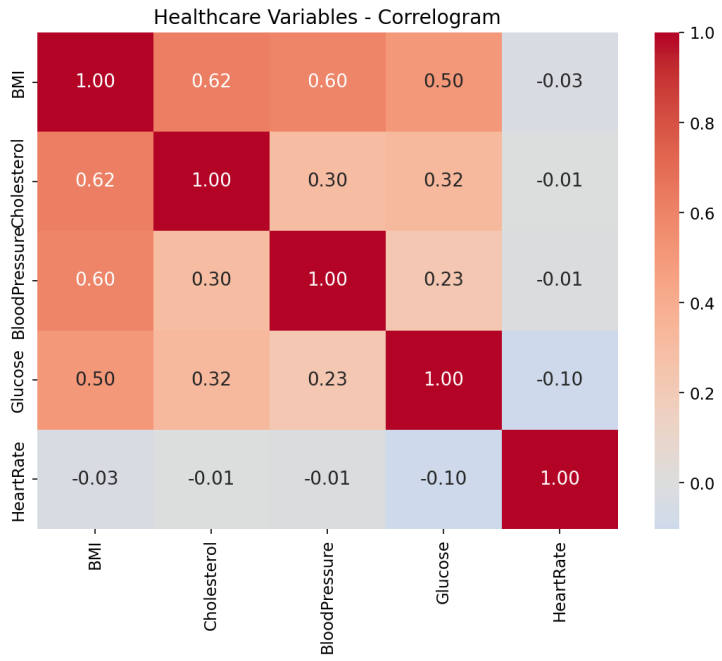


Q6
Healthcare Correlogram & Multicollinearity

Python Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# Simulated healthcare dataset
np.random.seed(4)
n = 200
bmi = np.random.normal(26, 4, n)
cholesterol = 150 + bmi * 3 + np.random.normal(0, 15, n)
blood_pressure = 100 + bmi * 1.2 + np.random.normal(0, 8, n)
glucose = 90 + bmi * 1.5 + np.random.normal(0, 10, n)
heart_rate = np.random.normal(72, 8, n)  # kept largely independent

df = pd.DataFrame({'BMI': bmi, 'Cholesterol': cholesterol,
                   'BloodPressure': blood_pressure, 'Glucose': glucose,
                   'HeartRate': heart_rate})

corr_matrix = df.corr()
print("Healthcare correlation matrix:\n", corr_matrix.round(3))

plt.figure(figsize=(7.5, 6.5))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, fmt='.2f')
plt.title("Healthcare Variables - Correlogram")
plt.tight_layout()
plt.show()
```

Output

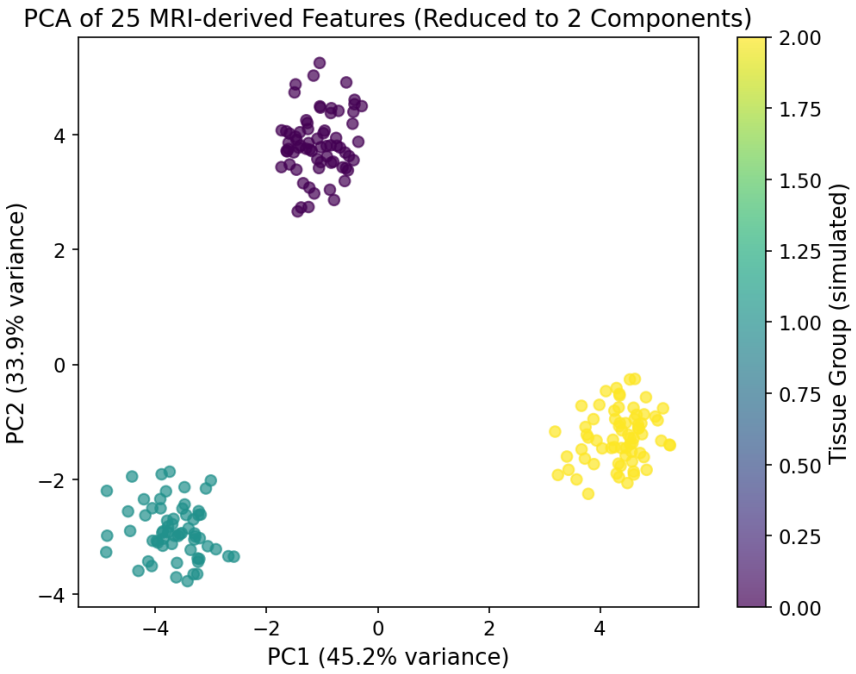


Q7
PCA on 25 MRI Features

Python Code

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler


# Simulated dataset: 25 features extracted from MRI images, for 200 scans,
# across 3 underlying "tissue type" groups (to give PCA something structured
# to reveal).
np.random.seed(5)
n_samples, n_features = 200, 25
groups = np.random.choice([0, 1, 2], n_samples)
group_centers = np.random.normal(0, 3, (3, n_features))
X = group_centers[groups] + np.random.normal(0, 1, (n_samples, n_features))


X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)


plt.figure(figsize=(7, 5.5))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=groups, cmap='viridis', alpha=0.7)
plt.title("PCA of 25 MRI-derived Features (Reduced to 2 Components)")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.colorbar(scatter, label='Tissue Group (simulated)')
plt.tight_layout()
plt.show()


print(f"Explained variance ratio: PC1={pca.explained_variance_ratio_[0]:.3f}, "
      f"PC2={pca.explained_variance_ratio_[1]:.3f}")
print(f"Total variance captured by 2 components: {pca.explained_variance_ratio_.sum()*100:.1f}%")
```

Output

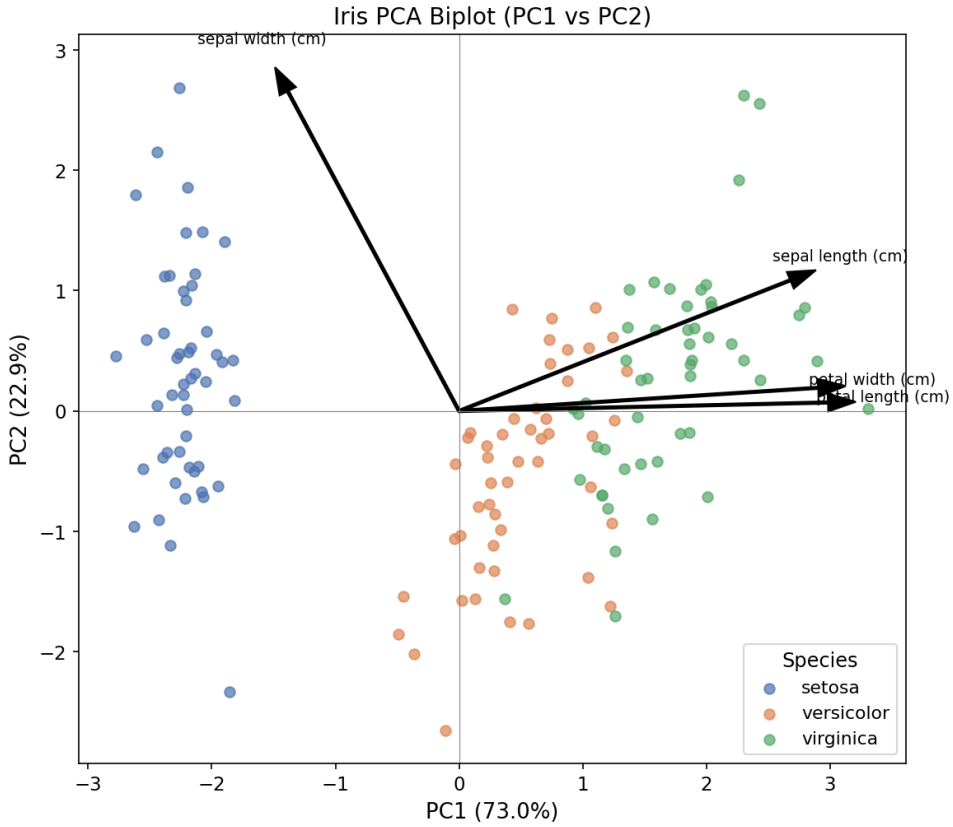


Q8
Iris PCA: Summary & Biplot

Python Code

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=4)
X_pca = pca.fit_transform(X_scaled)

# Summary (like R's summary(prcomp))
print("PCA Summary:")
print(f"{'Component':<12}{'Explained Var %':>18}{'Cumulative %':>16}")
cumulative = 0
for i, var in enumerate(pca.explained_variance_ratio_):
    cumulative += var
    print(f"PC{i+1:<11}{var*100:>17.2f}{cumulative*100:>16.2f}")

# Biplot: PC1 vs PC2 scores + loading vectors
plt.figure(figsize=(8, 7))
colors = ['#4C72B0', '#DD8452', '#55A868']
for i, target in enumerate(target_names):
    plt.scatter(X_pca[y == i, 0], X_pca[y == i, 1], color=colors[i], label=target, alpha=0.7)

loadings = pca.components_[:2].T * np.sqrt(pca.explained_variance_[:2])
scale = 3
for i, feature in enumerate(feature_names):
    plt.arrow(0, 0, loadings[i, 0] * scale, loadings[i, 1] * scale,
              color='black', width=0.02, head_width=0.15)
    plt.text(loadings[i, 0] * scale * 1.15, loadings[i, 1] * scale * 1.15,
             feature, fontsize=9, ha='center')

plt.title("Iris PCA Biplot (PC1 vs PC2)")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)")
plt.legend(title="Species")
plt.axhline(0, color='gray', linewidth=0.5)
plt.axvline(0, color='gray', linewidth=0.5)
plt.tight_layout()
plt.show()

# Which PC explains the most variance?
print(f"\nPC1 explains the highest variance: {pca.explained_variance_ratio_[0]*100:.1f}%, "
      f"driven mainly by petal length/width (longest loading vectors), which are the "
      f"most discriminating measurements between the three Iris species.")
```

Output

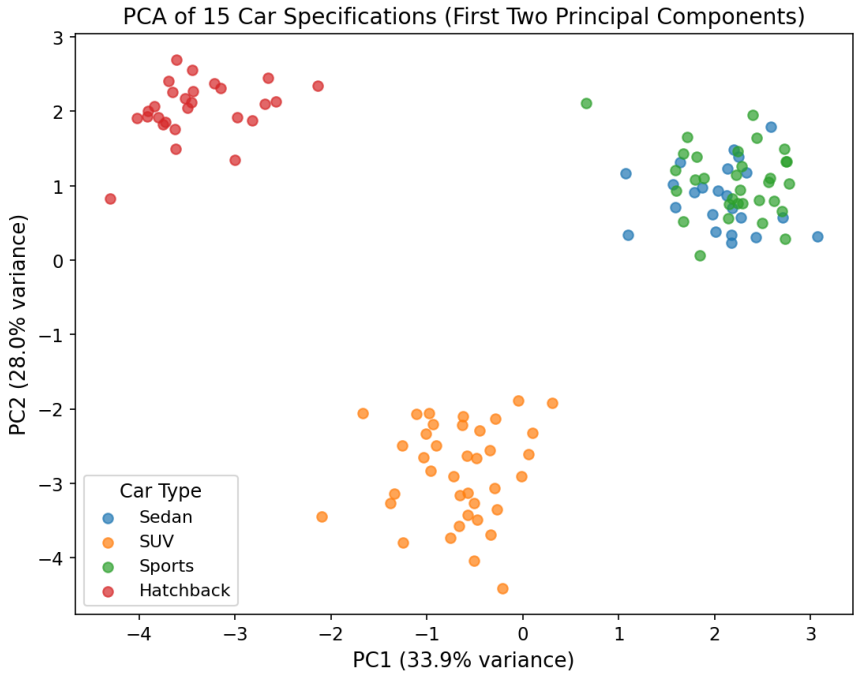

Iris PCA Biplot (PC1 vs PC2)
sepal width (cm)
sepal length (cm)
petal width (cm)
petal length (cm)
PC2 (22.9%)
PC1 (73.0%)
Species
setosa
versicolor
virginica

Q9
PCA on Automobile Specifications

Python Code

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Simulated dataset: 15 specifications for various cars (e.g. horsepower,
# weight, mpg, engine size, etc.), across 4 car categories
np.random.seed(6)
n_samples, n_features = 120, 15
categories = np.random.choice(['Sedan', 'SUV', 'Sports', 'Hatchback'], n_samples)
cat_idx = {c: i for i, c in enumerate(['Sedan', 'SUV', 'Sports', 'Hatchback'])}
cat_centers = np.random.normal(0, 2.5, (4, n_features))
X = np.array([cat_centers[cat_idx[c]] for c in categories]) + np.random.normal(0, 1, (n_samples, n_features))

X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(7.5, 6))
for cat in cat_idx:
    mask = categories == cat
    plt.scatter(X_pca[mask, 0], X_pca[mask, 1], label=cat, alpha=0.7)
plt.title("PCA of 15 Car Specifications (First Two Principal Components)")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.legend(title="Car Type")
plt.tight_layout()
plt.show()

print(f"Total variance captured by first 2 PCs: {pca.explained_variance_ratio_.sum()*100:.1f}%")
```

Output

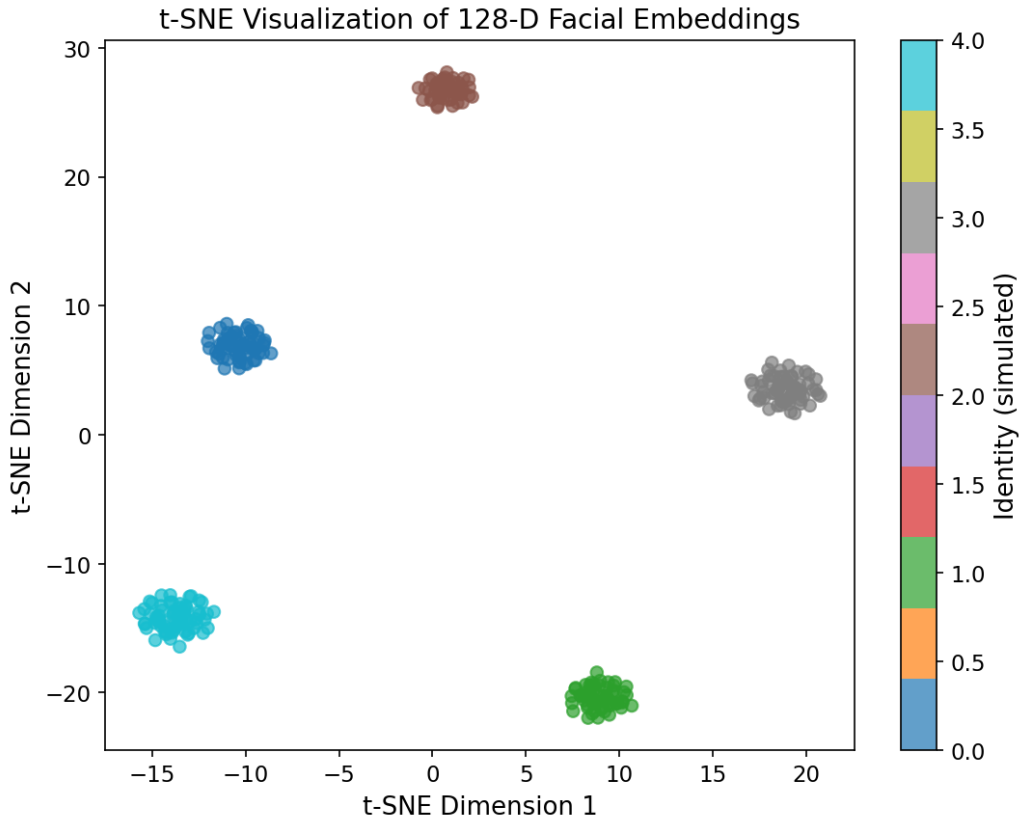


Q10
t-SNE on Facial Embeddings (128-D)

Python Code

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.manifold import TSNE

# Simulated dataset: 128-dimensional facial embeddings for 300 faces across
# 5 identities (a facial-recognition embedding is typically a compact vector
# where same-identity faces cluster tightly and different identities are
# well-separated).
np.random.seed(7)
n_samples, n_dims = 300, 128
identities = np.random.choice(range(5), n_samples)
identity_centers = np.random.normal(0, 5, (5, n_dims))
X = identity_centers[identities] + np.random.normal(0, 1, (n_samples, n_dims))

tsne = TSNE(n_components=2, perplexity=30, random_state=7)
X_tsne = tsne.fit_transform(X)

plt.figure(figsize=(7.5, 6))
scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=identities, cmap='tab10', alpha=0.7)
plt.title("t-SNE Visualization of 128-D Facial Embeddings")
plt.xlabel("t-SNE Dimension 1"); plt.ylabel("t-SNE Dimension 2")
plt.colorbar(scatter, label='Identity (simulated)')
plt.tight_layout()
plt.show()
```

Output

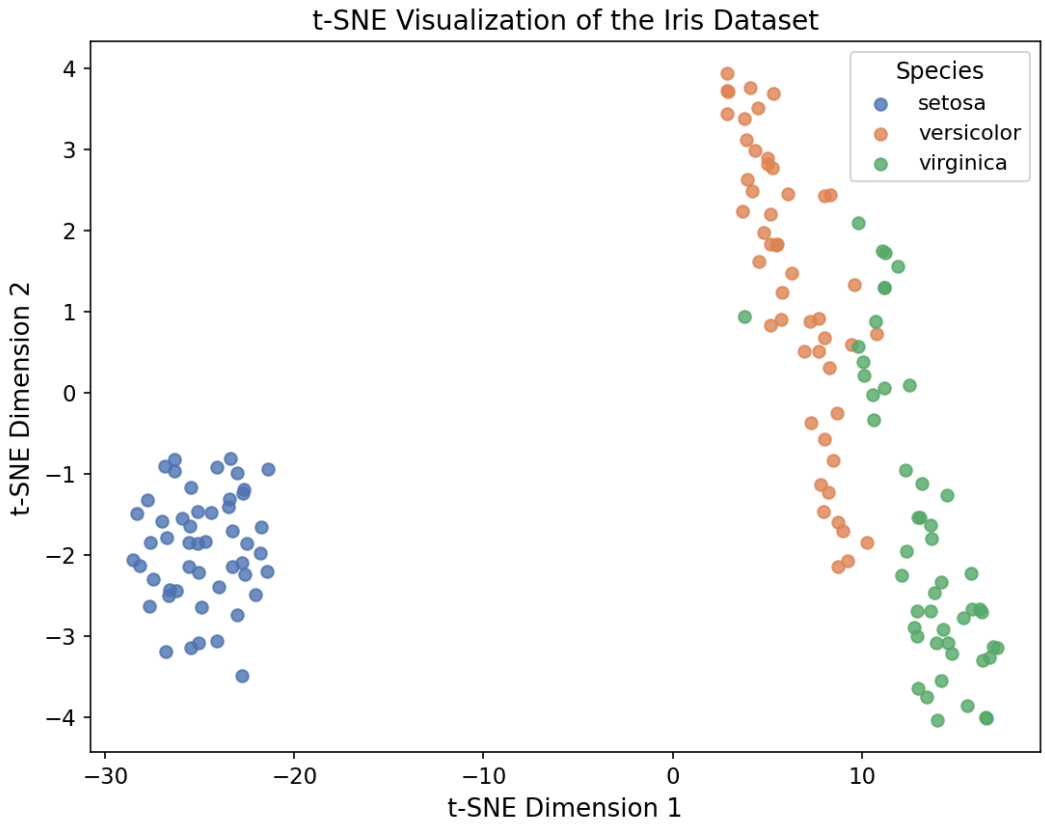


Q11
t-SNE on Iris Dataset

Python Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE

iris = load_iris()
X, y = iris.data, iris.target
target_names = iris.target_names

tsne = TSNE(n_components=2, perplexity=30, random_state=1)
X_tsne = tsne.fit_transform(X)

plt.figure(figsize=(7.5, 6))
colors = ['#4C72B0', '#DD8452', '#55A868']
for i, name in enumerate(target_names):
    plt.scatter(X_tsne[y == i, 0], X_tsne[y == i, 1], color=colors[i], label=name, alpha=0.8)
plt.title("t-SNE Visualization of the Iris Dataset")
plt.xlabel("t-SNE Dimension 1"); plt.ylabel("t-SNE Dimension 2")
plt.legend(title="Species")
plt.tight_layout()
plt.show()
```

Output

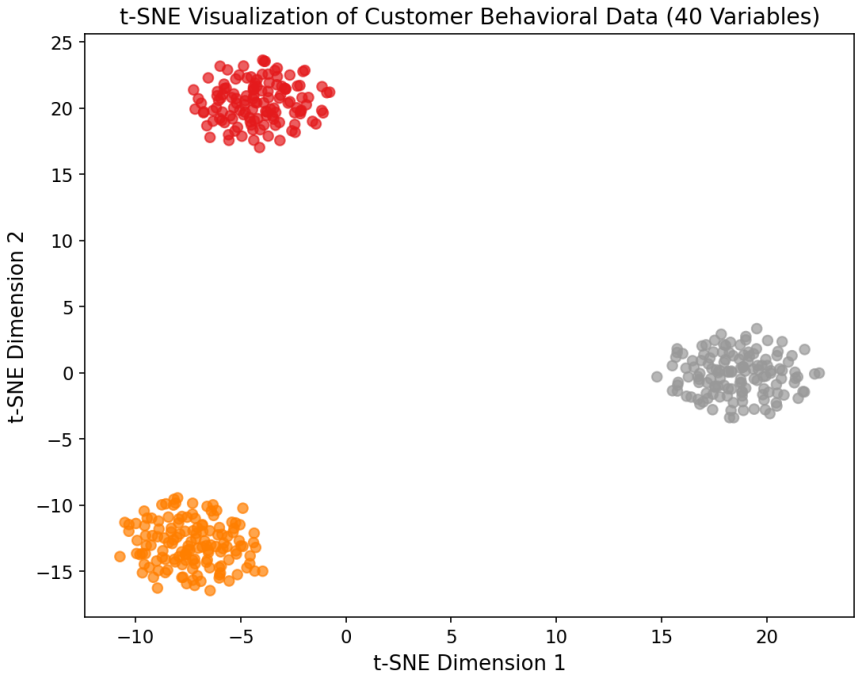


Q12
t-SNE on Customer Segmentation (40 Variables)

Python Code

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler

# Simulated dataset: 40 behavioral variables for 400 customers, drawn from
# 3 underlying (unlabeled, unknown-to-the-analyst) behavioral segments
np.random.seed(8)
n_samples, n_features = 400, 40
true_segments = np.random.choice(range(3), n_samples)  # for coloring only
segment_centers = np.random.normal(0, 4, (3, n_features))
X = segment_centers[true_segments] + np.random.normal(0, 1.5, (n_samples, n_features))
X_scaled = StandardScaler().fit_transform(X)

tsne = TSNE(n_components=2, perplexity=35, random_state=8)
X_tsne = tsne.fit_transform(X_scaled)

plt.figure(figsize=(7.5, 6))
scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=true_segments, cmap='Set1', alpha=0.7)
plt.title("t-SNE Visualization of Customer Behavioral Data (40 Variables)")
plt.xlabel("t-SNE Dimension 1"); plt.ylabel("t-SNE Dimension 2")
plt.tight_layout()
plt.show()

# Discussion: the t-SNE plot shows visually distinct, well-separated
# clusters of customers, indicating that the 40 behavioral variables do
# capture meaningful, separable customer segments rather than one
# undifferentiated blob. This supports treating customer segmentation as a
# genuine clustering problem (e.g. for targeted marketing) rather than
# assuming customers form a single homogeneous group.
```

Output

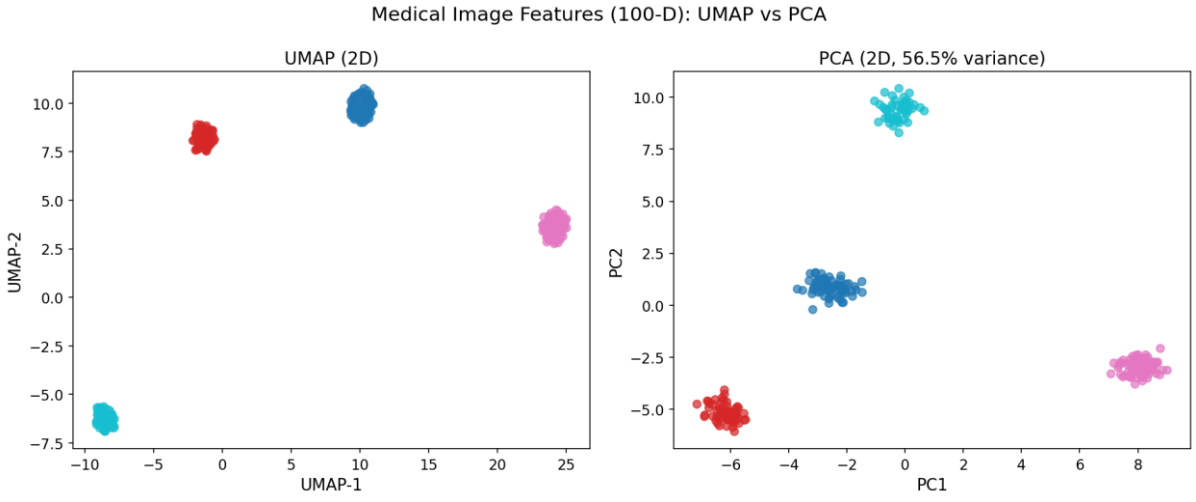


Q13
UMAP on Medical Image Features (vs PCA)

Python Code

```
import matplotlib.pyplot as plt
import numpy as np
import umap
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Simulated dataset: 100 extracted features from a medical image dataset,
# across 4 diagnostic categories
np.random.seed(9)
n_samples, n_features = 250, 100
categories = np.random.choice(range(4), n_samples)
cat_centers = np.random.normal(0, 3, (4, n_features))
X = cat_centers[categories] + np.random.normal(0, 1, (n_samples, n_features))
X_scaled = StandardScaler().fit_transform(X)

reducer = umap.UMAP(n_components=2, random_state=9)
X_umap = reducer.fit_transform(X_scaled)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
sc0 = axes[0].scatter(X_umap[:, 0], X_umap[:, 1], c=categories, cmap='tab10', alpha=0.7)
axes[0].set_title("UMAP (2D)")
axes[0].set_xlabel("UMAP-1"); axes[0].set_ylabel("UMAP-2")

sc1 = axes[1].scatter(X_pca[:, 0], X_pca[:, 1], c=categories, cmap='tab10', alpha=0.7)
axes[1].set_title(f"PCA (2D, {pca.explained_variance_ratio_.sum()*100:.1f}% variance)")
axes[1].set_xlabel("PC1"); axes[1].set_ylabel("PC2")

plt.suptitle("Medical Image Features (100-D): UMAP vs PCA")
plt.tight_layout()
plt.show()
```

Output



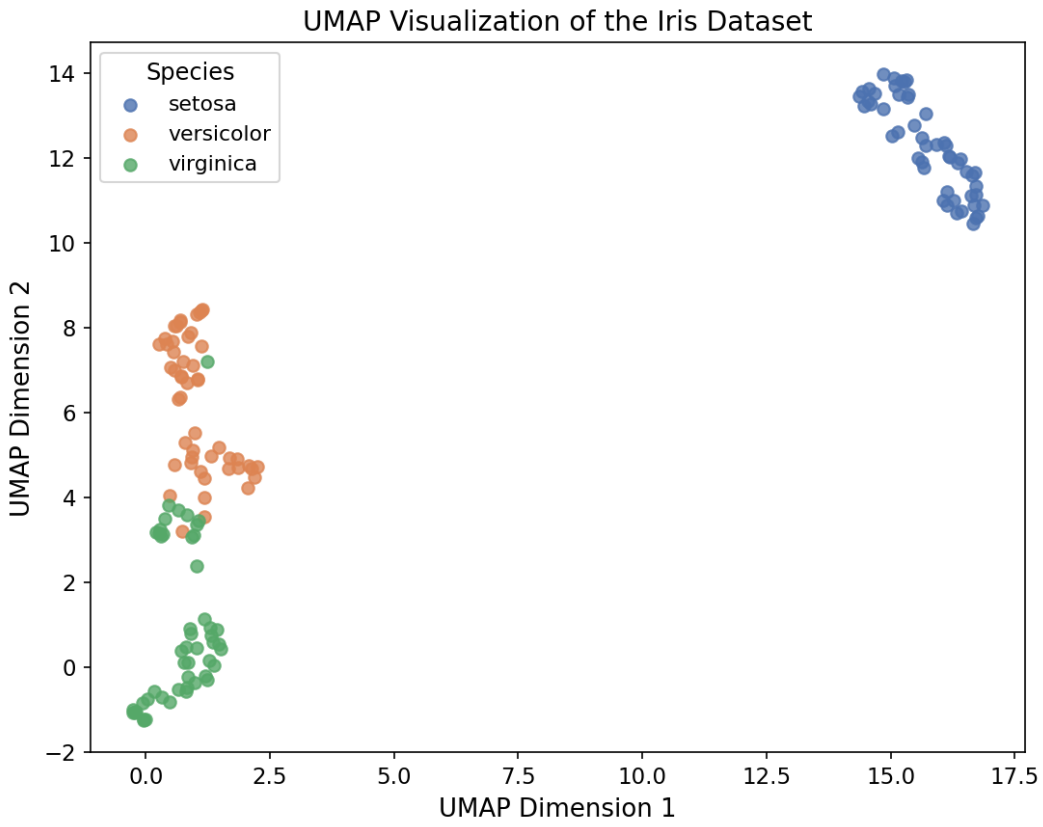
Q14

UMAP on Iris Dataset

Python Code

```
import matplotlib.pyplot as plt
import umap
from sklearn.datasets import load_iris

iris = load_iris()
X, y = iris.data, iris.target
target_names = iris.target_names

reducer = umap.UMAP(n_components=2, random_state=1)
X_umap = reducer.fit_transform(X)

plt.figure(figsize=(7.5, 6))
colors = ['#4C72B0', '#DD8452', '#55A868']
for i, name in enumerate(target_names):
    plt.scatter(X_umap[y == i, 0], X_umap[y == i, 1], color=colors[i], label=name, alpha=0.8)
plt.title("UMAP Visualization of the Iris Dataset")
plt.xlabel("UMAP Dimension 1"); plt.ylabel("UMAP Dimension 2")
plt.legend(title="Species")
plt.tight_layout()
plt.show()
```

Output



Q15
UMAP on E-Commerce Purchasing Behavior

Python Code

```
import matplotlib.pyplot as plt
import numpy as np
import umap
from sklearn.preprocessing import StandardScaler

# Simulated dataset: 60 purchasing-behavior attributes for 350 customers,
# across 5 underlying (unlabeled) customer clusters
np.random.seed(10)
n_samples, n_features = 350, 60
clusters = np.random.choice(range(5), n_samples)  # for coloring only
cluster_centers = np.random.normal(0, 4, (5, n_features))
X = cluster_centers[clusters] + np.random.normal(0, 1.5, (n_samples, n_features))
X_scaled = StandardScaler().fit_transform(X)

reducer = umap.UMAP(n_components=2, random_state=10)
X_umap = reducer.fit_transform(X_scaled)

plt.figure(figsize=(7.5, 6))
scatter = plt.scatter(X_umap[:, 0], X_umap[:, 1], c=clusters, cmap='tab10', alpha=0.7)
plt.title("UMAP Visualization of E-Commerce Purchasing Behavior (60 Attributes)")
plt.xlabel("UMAP Dimension 1"); plt.ylabel("UMAP Dimension 2")
plt.tight_layout()
plt.show()
```

Output